

صيانة البرمجيات وارتقاءها وإدارتها - Software Maintenance, Evolution and Management

هدف الفصل

يهدف هذا الفصل إلى:

- 1- شرح أهمية التغيير وكيف أنه لا يمكن تجنبه إذا أردنا أن تبقى البرمجيات مفيدة.
- 2- مناقشة صيانة البرمجيات والعوامل التي تؤثر على تكاليفها.
- 3- وصف الإجراءات المعنية بتطور وارتقاء البرمجيات software evolution.
- 4- مناقشة استراتيجيات ضمان تطور الأنظمة.
- 5- تعرف إدارة التغيير.
- 6- تعرف إدارة التشكيلات والإصدارات والسحوب.

1. التغيير في البرمجيات

إن من المشاكل الهامة التي تواجه المؤسسات هي إدارة التغيير وتنفيذ تغييرات على الأنظمة البرمجية الخاصة بها. لكن من غير الممكن تجنب تغيير وتطوير البرمجيات خلال استخدامها وإلا فإنها ستتكون مفيدة لمرحلة مؤقتة ثم تصبح غير متناسبة مع الأعمال. يمكن أن نذكر عدة أسباب للتغيير في البرمجيات منها:

1- ظهور متطلبات جديدة في الأعمال.

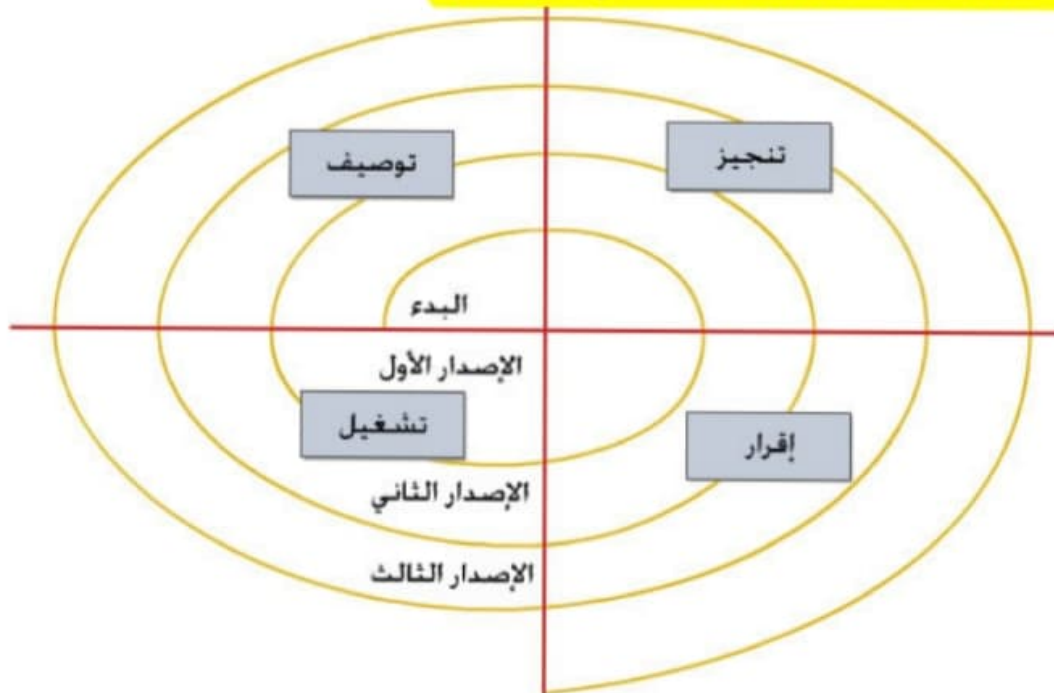
2- تغير بيئة العمل.

3- ظهور أخطاء تحتاج إلى تصحيح.

4- إضافة حواسيب وتجهيزات جديدة إلى النظام.

5- ضرورة تحسين أداء النظام أو وظيفته.

كما أن أهمية التغيير والتعديل في الأنظمة البرمجية تأتي من أن هذه الأنظمة تعتبر من الأصول الهامة في أرصدة الأعمال ومن أن المؤسسات تضع استثمارات ضخمة فيها. لذلك ليس من المستغرب أن نجد أن معظم الميزانية المخصصة للبرمجيات في الشركات الكبيرة تخص لترقية وتحسين البرمجيات الموجودة بدلاً من تطوير برمجيات جديدة. يمكننا بالنتيجة التفكير في بهندسة البرمجيات كإجرائية حازونية تمتد على مدى حياة النظام (انظر الشكل).



2. ديناميكية ارتقاء البرامج

ديناميكية ارتقاء البرامج هي دراسة تغييرات الأنظمة. بعد دراسات تجريبية هامة قام بهـ اـكـلـمـنـ Belady و Lehman اقترحا مجموعة من القوانين التي تنطبق على جميع الأنظمة عند تغييرها وترقيتها (انظر الجدول: قوانين Lehman).

القانون	الوصف
تغيير مستمر	إن برنامجاً مستخدماً في بيئة حقيقية يجب أن يتغير أو أنه سيـصبح مـع الزمن أقل فائدة في تلك البيئة.
تعقيد متزايد	عندما يتغير برنامج ما ويرتقي تصبح بنيته أكثر تعقيداً. يجب بـتخـصيص موارد إضافية للحفاظ على البنية وتبسيطها.
ارتقاء كبير للبرامج	ارتقاء البرامج هي إجرائية ذاتية التصحيح. فمواصفات النظام كـالحجم والفترة الزمنية بين الإصدارات وعدد الأخطاء هي تقريباً ثابتة فـي كـل إصدار.
استقرار مؤسسي	نسبة التطوير ثابتة تقريباً على مدى حياة نظام ما ومـستقلة عـن المـوارد المخصصة لذلك.
انحفاظ الألفة	التغيير المتزايد في كل إصدار ثابت تقريباً على مدى حياة النظام.
تضخم مستمر	الوظائف المقدمة من قبل نظام ما يجب أن تزيد باستمرار للحفاظ على رضا المستخدمين.
جودة متناقصة	ستبدو الجودة متناقصة إلا إذا ساءت الأنظمة التغيير في بيئة التشغيل.
نظام تغذية راجعة	تتضمن إجراءات الارتقاء أنظمة تغذية راجعة يجب الاهتمام بها كمؤشرات هامة عن تحسين المنتج.

يمكن اعتبار هذه القوانين ملاحظات دقيقة تُطبق غالباً على الأنظمة الكبيرة في المؤسسات الـضخمة. وليس من الواضح كيفية ملائمتها للبرامج الأخرى الصغيرة أو المتوسطة أو التي تتـضمن مكوـنات جاهزة أو التي تُنفذ في مؤسسات صغيرة.

3. صيانة البرمجيات

صيانة البرمجيات هي الإجرائية العامة لتغيير البرمجيات والتعديل فيها بعد وضعها في بيئة العمل والاستخدام. لا تتضمن الصيانة عادة تغييرات كبيرة في بنية النظام بل تكون تغييرات في بعض المكونات الموجودة أو إضافة مكونات جديدة.

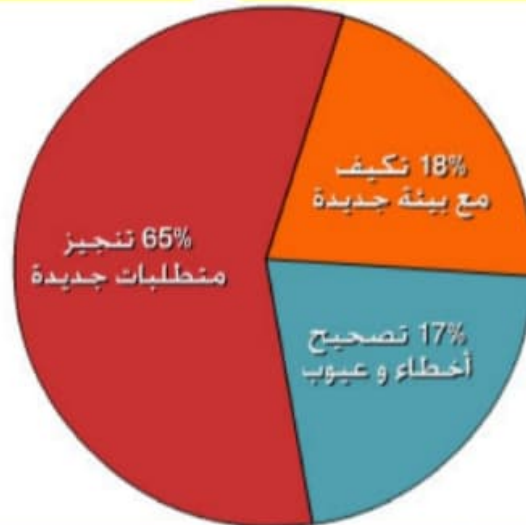
لا يمكن تجذب أعمال الصيانة للبرمجيات فمتطلبات النظام تتغير أثناء استخدامه، وبيئة العمل في تغير دائم مما يجعل استمرار الاستفادة من النظام مرهوناً بصيانتته المستمرة.

هنالك ثلاثة أنواع من الصيانة البرمجية:

- 1- تصحيح أخطاء برمجية أو إصلاح عيوب في النظام أو ملاقة متطلبات غير متوفرة.
- 2- تكيف النظام مع بيئة تشغيل تختلف عن بيئة تشغيله الأولى (حواسيب، نظم تشغيل، ...).
- 3- تعديل في وظائف النظام أو إضافة وظائف جديدة له تعكس متطلبات جديدة.

3. صيانة البرمجيات

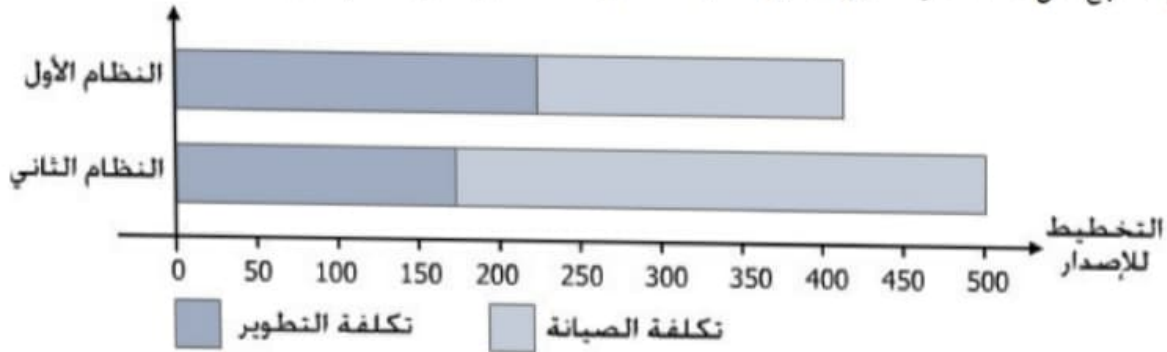
أظهرت مجموعة من الدراسات الإحصائية أن حوالي 65% من الصيانة تتعلق بتنفيذ متطلبات جديدة بينما 18% من الصيانة يكون للتكيف مع بيئة جديدة و 17% لتصحيح أخطاء و عيوب.



تكون تكلفة صيانة البرمجيات عادة أكبر بكثير من تكلفة تطوير البرمجيات وقد تكون ضئيلة. هذه التكلفة قد تصل إلى مائة ضعف حسب نوع التطبيق. تتزايد هذه التكلفة باستمرار الصيانة خاصة إذا كانت تلك الصيانة تؤثر على بنية النظام مما يجعل صيانتها المستقبلية أكثر صعوبة. كذلك فإن الأنظمة القديمة المعمرة تصبح تكلفة صيانتها كبيرة مثل الأنظمة التي تستخدم تجهيز ذات قيمة أو المكتوبة بلغات برمجة قديمة.

3. صيانة البرمجيات

من المجدي دوماً استثمار جهد أكبر في التصميم والتنجز للخروج بنظام واضح وسهل الفهم والتغيير - مما يساهم في تقليل تكاليف الصيانة. يظهر الشكل التالي كيف أن التكلفة العامة للنظام على مدى حياته يمكن أن تصبح أقل عند صرف جهد أكبر خلال التطوير للحصول على نظام سهل الصيانة.



3. صيانة البرمجيات

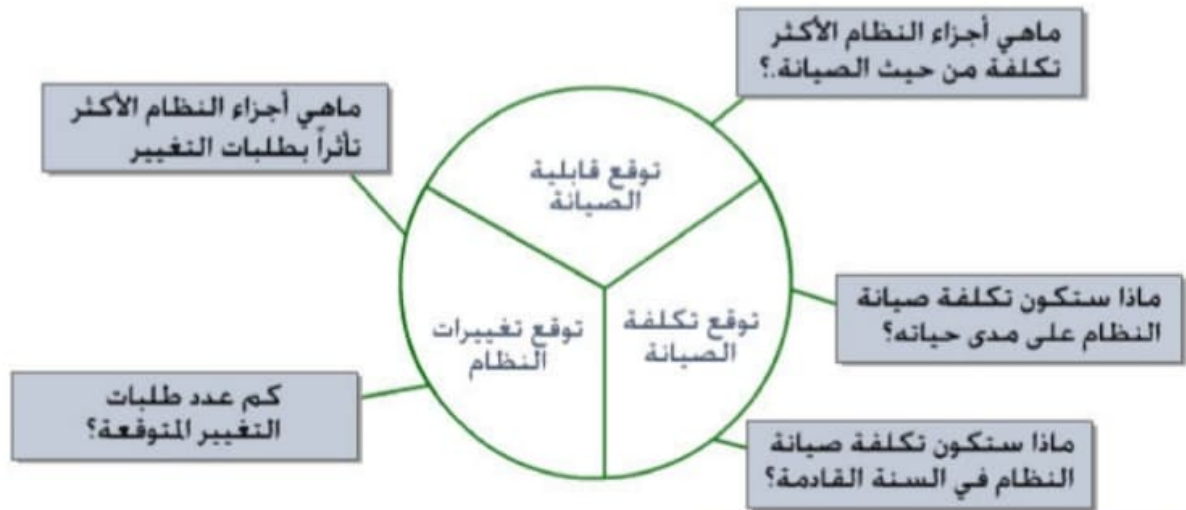
هناك مجموعة من العوامل التي تؤثر على تكلفة الصيانة منها:

- 1- استقرار الفريق: تتأثر تكاليف الصيانة كثيراً باستمرار عمل نفس الفريق - ق فيه - ا، فالعناصر - ر الجديدة ستتطلب جهداً ووقتاً كبيراً لفهم النظام وتنجز تغيرات عليه.
- 2- المسؤولية التعاقدية: قد لا تكون الجهة المطورة هي نفسها المسؤولة عن الصيانة مما يجعل - ل لديها أولويات أخرى في توفير الجهود أثناء التطوير على حساب تكلفة عالية للصيانة.
- 3- مهارات فريق الصيانة: يكون فريق الصيانة غالباً قليل الخبرة ولديه معرفة محد - دودة بمج - ال العمل.
- 4- تعمير البرامج وبنيتها: مع تقدم البرامج في العمر تصبح بنيتها متشعبة فتصبح صعبة الفهم - م والصيانة.

3. صيانة البرمجيات

1- توقع الصيانة

يكره المدراء عادة المفاجآت خاصة تلك المكلفة منها. يجب إذا توقع التغيرات على النظام - ام وتحدي - د الأجزاء التي ستكون صعبة الصيانة فيه. كما يجب تقدير تكاليف صيانة النظام في فترة زمنية معينة - ة. يظهر الشكل التالي هذه التوقعات مع الأسئلة المرافقة لها.



هذه التوقعات والتقديرات مترابطة بشدة:

- 1- يعتمد قبول التغيير على قابلية صيانة المكونات التي تتأثر بهذا التغيير.
- 2- تنجيز التغييرات يشتت النظام ويقلل من قابليته للصيانة.
- 3- تعتمد تكلفة الصيانة على عدد التغييرات وتعتمد تكلفة التغيير على قابلية الصيانة.

2- توقع التغيير

يتطلب توقع عدد التغييرات فهماً عميقاً للعلاقة بين النظام والبيئة التي يعمل فيها فالأنظمة المرتبطة بشدة بالبيئة تتطلب تغييراً بتغييرها. من العوامل التي تؤثر على هذه العلاقة:

- 1- عدد واجهات النظام ودرجة تعقيدها.
- 2- عدد متطلبات النظام ذات الطبيعة المتغيرة.
- 3- عدد إجراءات الأعمال التي تستخدم النظام.

لتوقع التغيير والصيانة يجب فهم عدد ونوعية العلاقات بين مكونات النظام ودرجة تعقيدها. أثبتت الدراسات الإحصائية أن معظم جهود الصيانة تتركز على عدد قليل من مكونات النظام. يمكن اعتماد مجموعة من القياسات للتعقيد مثل:

- 1- تعقيد بنى التحكم.
- 2- تعقيد بنى المعطيات.
- 3- حجوم الأغراض والإجراءات والطرائق والمجتمعات.

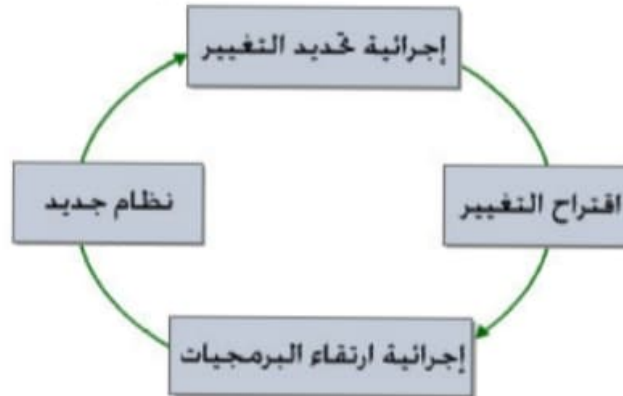
بعد وضع النظام في الاستخدام يمكن استخدام معطيات الإجراءات لتوقع الصيانة، فيم. ا. ي. بع. ض. الأمثلة عن المقاييس التي يمكن أن تُستخدم لذلك:

- 1- عدد طلبات الصيانة التصحيحية.
- 2- الزمن الوسطي اللازم لتحليل تأثيرات الصيانة.
- 3- الزمن الوسطي اللازم لتنفيذ طلب تغيير.
- 4- عدد طلبات التغيير المنتظرة.

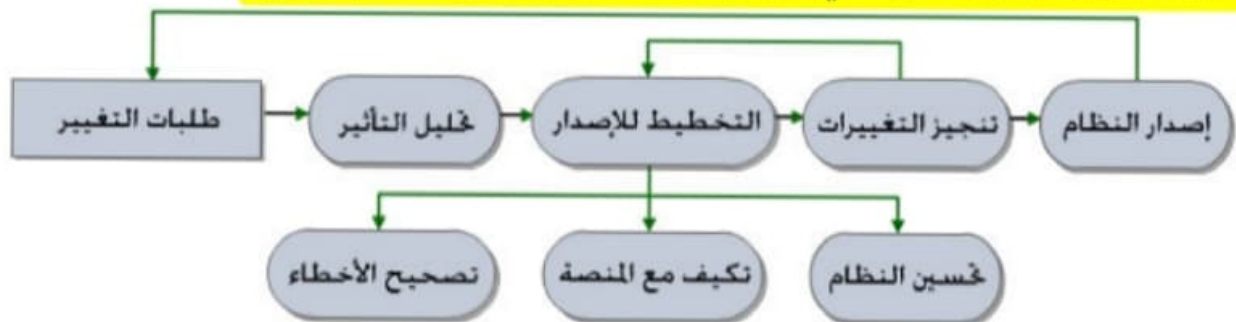
إن تزايد أي من القياسات السابقة يدل على تناقص في قابلية الصيانة.

4. إجراءات الارتقاء

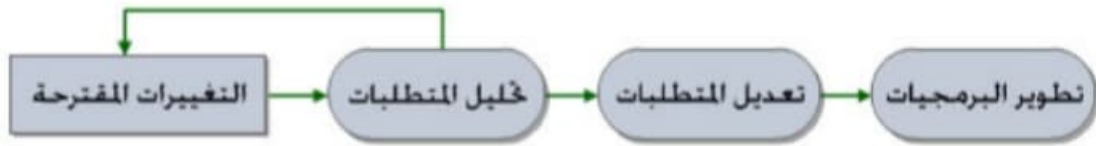
تعتمد إجراءات الارتقاء على نوعية البرمجيات التي تجدي صيانتها، وعلى إجراءات التطوير المستخدمة، ومهارات فريق العمل وخبراته. في بعض المؤسسات يمكن أن يكون التطور غير رسمياً حيث تأتي التغييرات نتيجة مناقشات بين مستخدمي النظام ومطوريه. في مؤسسات أخرى قد تكون له صفة رسمية موثقة. في جميع الأحوال يكون المحرك الدافع للارتقاء هو اقتراحات التغيير. تبدأ العملية من تحديد التغييرات ثم اقتراحها وتستمر خلال فترة حياة النظام (انظر الشكل).



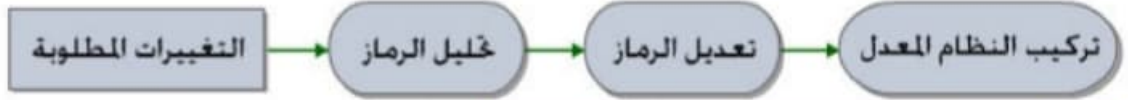
تتضمن إجراءات الارتقاء مجموعة من النشاطات الأساسية أهمها تحليل التغيير والتخطيط للإصدار الجديد وتنفيذه. يظهر الشكل التالي مراحل ونشاطات إجراءات ارتقاء نظام برمجي.



في الحالة المثالية تؤدي مرحلة تغيير التنفيذ من هذه الإجراءات إلى تغيير التوصيف والتصميم بما يتوافق مع التغييرات الجديدة (انظر الشكل).



يمكن أن تتعلق طلبات التغيير بمشاكل في النظام يجب معالجتها بسرعة، دون المرور بمراحل إجرائية هندسة البرمجيات حيث يصبح مخطط العمل كما في الشكل التالي.



يمكن أن تظهر التغييرات الإسعافية هذه لثلاثة أسباب:

- 1- خطأ حقيقي هام في النظام يجب تصحيحه.
- 2- تغييرات في بيئة النظام لها تأثيرات غير متوقعة (مثل ترقية نظام التشغيل).
- 3- تغييرات في الأعمال تتطلب استجابة سريعة (كإصدار منتج منافس مثلاً).

5. إعادة هندسة الأنظمة

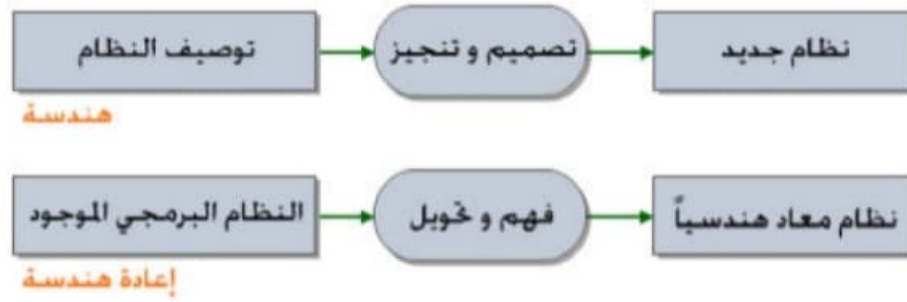
إن الارتقاء بنظام ما يتطلب فهم البرنامج الذي يجب تغييره قبل إنجاز التغيير. غالباً ما تكون الأنظمة القديمة صعبة الفهم لكونها بنيت على أساس زيادة الأداء وتقليل حجوز التخزين. لتبسيط عملية تغيير الأنظمة القديمة تلجأ بعض الشركات إلى إعادة هندسة النظام، أي إعادة بناءه أو إعادة كتابة أجزاء منه دون تغيير وظائفه، ليصبح بعد ذلك قابلاً للصيانة. يمكن أن تتضمن إعادة الهندسة إعادة التوجيه والتوثيق أو إعادة التنظيم والبناء أو كتابة البرمجيات بلغة حديثة.

تطبق إعادة الهندسة عندما تكون بعض أجزاء منظومة برمجية كبيرة تتطلب صيانة مستمرة ومتكررة. حيث يصبح من المفيد إعادة كتابة هذه الأجزاء لزيادة قابلية صيانتها. تتميز إعادة الهندسة عن الطرائق

الأخرى للارتقاء بالبرمجيات بفائدتين رئيسيتين:

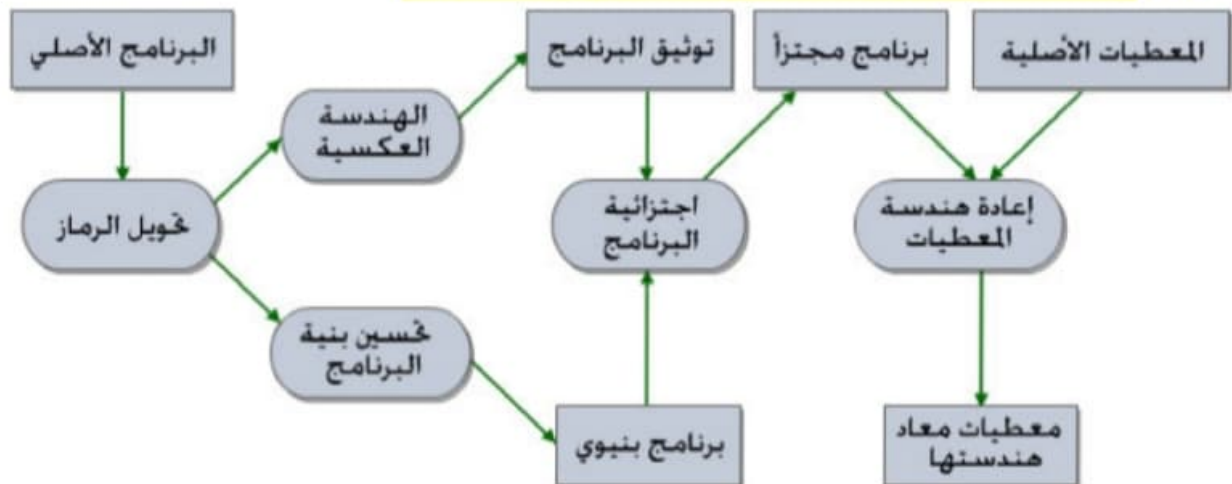
- 1- **مخاطرة أقل:** يرتبط تطوير البرمجيات الجديدة دوماً بمخاطر كبيرة تتعلق بالتطوير وفريق العمل ومشاكل التوصيف وعوامل أخرى كثيرة، بينما تكون إعادة الهندسة أقل مخاطرة.
- 2- **تكلفة أقل:** تكلفة إعادة الهندسة تكون غالباً أقل بكثير من تكلفة تطوير برمجيات جديدة.

الفرق الهام بين إعادة هندسة نظام ما وتطوير نظام جديد هو نقطة البداية. فبدلاً من البدء من توصيفات مكتوبة يجري عند إعادة الهندسة البدء من نظام برمجي موجود (انظر الشكل).



يظهر الشكل التالي النشاطات التي تتضمنها إجرائية إعادة الهندسة:

- 1- تحويل الرماز المصدر: تحويل الرماز وكتابته بلغة جديدة أو نسخة جديدة من اللغة الأصلية.
- 2- الهندسة العكسية: تحليل النظام بهدف فهم بنيته ووظائفه.
- 3- تحسين بنية البرامج: تحليل بنية التحكم وجعلها سهلة الفهم.
- 4- اجتزائية البرامج: إعادة تنظيم بنية البرامج بتجميع الأجزاء عند مناسبة ذلك وبإلغاء التكرار عند وجوده. ويمكن أن يتضمن ذلك التحويل من بنية مركزية إلى بنية موزعة.
- 5- إعادة هندسة المعطيات: تنظيف وإعادة تنظيم بنية المعطيات.



تعتمد تكلفة إعادة الهندسة على الجهد المبذول لذلك. فهناك طيف واسع من الطرق المستخدمة لـ ذلك. تتغير الكلفة بتغير هذه الطرق (انظر محور التكلفة في الشكل التالي).



هناك عدة عوامل مؤثرة في تكلفة إعادة هندسة البرمجيات:

- 1- جودة البرمجيات التي سيجري إعادة هندستها.
- 2- الأدوات المتوفرة لدعم عملية إعادة الهندسة.
- 3- المدى المطلوب لتحويل وتصحيح المعطيات.
- 4- توفر فريق عمل خبير في إعادة الهندسة.

6. الارتقاء بالأنظمة القيمة

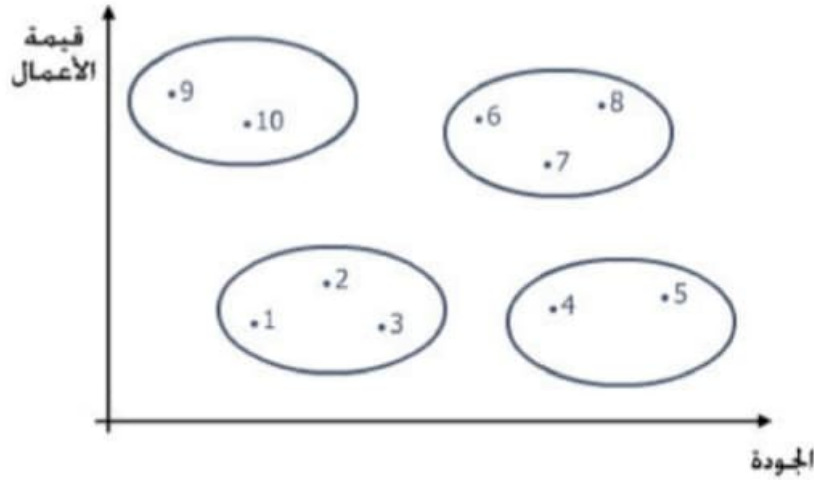
يجب على المؤسسات التي تعتمد على أنظمة قديمة أن تختار استراتيجية مناسبة للارتقاء بهذه الأنظمة وتطويرها من بين الاستراتيجيات التالية:

- 1- إلغاء النظام نهائياً وتغيير إجراءات الأعمال بحيث لا تكون هناك حاجة له.
- 2- الاستمرار في صيانة النظام.
- 3- تحويل النظام وإعادة هندسته لتحسين قابليته للصيانة.
- 4- استبدال النظام بنظام جديد.

تتعلق الاستراتيجية المختارة بجودة النظام وأهميته وقيمته في الأعمال. لتوضيح ذلك سنفترض مؤسسة لديها 10 أنظمة قديمة جرى تحديد جودة وقيمة كل منها ثم جرى تمثيلها في مخطط بالـ شكل التالي. نلاحظ من الشكل أن هناك 4 أصناف من الأنظمة:

- 1- جودة منخفضة وقيمة أعمال منخفضة: هذه الأنظمة يجب إلغاؤها (1، 2، 3).
- 2- جودة منخفضة وقيمة أعمال مرتفعة: مساهمة كبيرة في الأعمال لكن تكاليف صيانة كبيرة. يجب إعادة هندسة هذه الأنظمة أو استبدالها إذا توفر نظام مناسب بديل (9، 10).
- 3- جودة عالية وقيمة أعمال منخفضة: يمكن استبدالها بمنتجات جاهزة إذا توفرت، أو إلغاؤها كلياً أو الاستمرار بصيانتها (4، 5).

- 4- جودة عالية وقيمة أعمال مرتفعة: يمكن استمرار التشغيل باستخدام الصيانة الطبيعية للنظام (6، 7، 8).



يجب أن يتناول تحديد قيمة الأعمال لنظام معين عدة وجهات نظر منها:

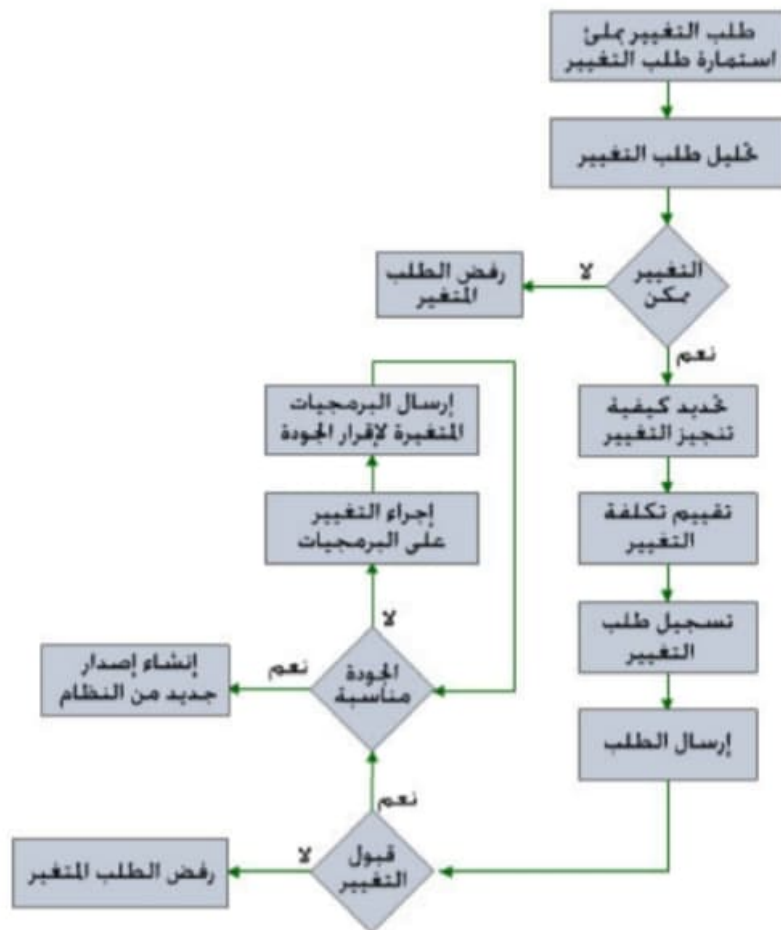
- 1- المستخدمون النهائيون للنظام.
- 2- زبائن المؤسسة.
- 3- مدراء خطوط التشغيل.
- 4- مدراء التقنية.
- 5- المدراء في المستوى الأعلى للمؤسسة.

أما تقييم الجودة فيتعلق بثلاثة نواحي هامة:

- 1- تقييم إجراءات الأعمال: إلى أي درجة تدعم إجراءات الأعمال الأهداف الحالية للمؤسسة.
- 2- تقييم البيئة: مدى فعالية البيئة الحالية وما تكاليف صيانتها.
- 3- تقييم التطبيق: ما هي جودة النظام البرمجي للتطبيق.

7. إدارة التغيير

تهتم إدارة التغيير بتسجيل التغييرات التي تجري على النظام بطلب من المستخدمين أو المطورين أو مدافع عن التغييرات السوق وضمان تنفيذها بأفضل طريقة من حيث الجدوى والتكلفة. هناك إجراءات معروفة ومحددة لإدارة التغيير تبدأ من تحليل التكلفة والفائدة للتغييرات المقترحة ثم تحديد المكونات التي سيجري تغييرها (انظر إجراءات إدارة التغيير في الشكل التالي).



1- طلب التغيير بملئ استمارة طلب التغيير

2- تحليل طلب التغيير

3- إذا كان التغيير ممكناً

1-3- تحديد كيفية تنجيز التغيير

2-3- تقييم تكلفة التغيير

3-3- تسجيل طلب التغيير في قاعدة المعطيات

4-3- إرسال الطلب إلى هيئة إدارة التغيير

5-3- عند قبول التغيير

3-5-1- تكرار

3-5-1-1- إجراء التغيير على البرمجيات

3-5-1-2- إرسال البرمجيات المتغيرة لإقرار الجودة

3-5-2- إلى أن تكون الجودة مناسبة

3-5-3- إنشاء إصدار جديد من النظام

3-6- في حال عدم قبول التغيير

3-6-1- رفض طلب التغيير

4- فيما عدا ذلك

4-1- رفض طلب التغيير

يجري طلب التغيير باستمارة خاصة يملؤها طالب التغيير تتضمن التغيير المقترح وطالب التغيير وأسباب التغيير ومدى العجلة المطلوبة لإجرائه. كما تتضمن تقييم التغيير، وتكلفته والتوصيات من قبل فريق الصيانة.

من المهم جداً تسجيل حالات التغيير. توفر الأدوات المساعدة في هندسة البرمجيات أدوات خاصة لإدارة التغيير، تقوم بتسجيل حالات طلبات التغيير وترسلها إلى الأشخاص المعنيين في الوقت المناسب، يمكن مكاملة هذه الأدوات مع البريد الإلكتروني لتوزيع طلبات التغيير مباشرة.

يجب مراجعة التغييرات من قبل مجموعة خارجية مستقلة عن إدارة المشروع يمكن أن تضم ممثلي الزبائن وأطراف العقد، تُقرر من وجهة نظر استراتيجية مؤسساتية جدوى التغيير مقابل التكلفة.

يكون هنالك عادة مع كل مكون من مكونات النظام سواء كان برنامج أو وثيقة سجل تاريخي للتغييرات. يسجل في هذا السجل التغييرات المجراة وأسبابها ومن الذي قام بها ومتى. يمكن وضع هذه المعلومات كملاحظات على الرماز (انظر المقطع البرمجي في الشكل التالي). كما يمكن الاستفادة منها ومعالجتها آلياً بواسطة أدوات برمجية إذا كانت تتبع أسلوب كتابة ثابت ومحدد.

```
// BANKSEC project (IST 6087)
//
// BANKSEC-TOOLS/AUTH/RBAC/USER_ROLE
//
// Object: currentRole
// Author: N. Perwaiz
// Creation date: 10th November 2002
//
// © Lancaster University 2002
//
// Modification history
// Version      Modifier Date      Change      Reason
// 1.0      J. Jones      1/12/2002      Add header      Submitted to CM
// 1.1      N. Perwaiz      9/4/2003      New field      Change req. R07/02
```

8. إدارة التشكيلات والإصدارات والسحوب Configuration, Version and Release management

مع تطور النظام وارتفاعه وحصول تغييرات عليه يجري إنشاء إصدارات جديدة منه:

1- لأنواع مختلفة من الحواسيب أو أنظمة التشغيل.

2- لتقديم وظائف جديدة.

3- لملاءمة استخدام معين لزيون معين.

تهتم إدارة التشكيلات بإدارة التطورات التي تطرأ على النظام، فهي عمل جماعي للفريق هدفه التحكم بالتكلفة والجهد المرافق لإجراء التغييرات على النظام. يتضمن ذلك تطوير إجراءات خاصة وتطبيقها. وقد يكون ضمن مجال إدارة الجودة. يمكن اعتماد معايير لتحديد كيفية تسمية الإصدارات والتحكم بالتغييرات وإدارة الإصدارات الجديدة.

قد يكون من الضروري إدارة كافة منتجات البرمجية م- ن توصف بصفات وصالحية وب- رامج ومعطيات اختبار وأدلة استخدام. يمكن أن يكون هنالك آلاف الوثائق المولدة لنظام برمجي كبير ومعقد. يجب وضع خطة لإدارة التشكيلات تتضمن هذه الخطة:

1- أنواع الوثائق التي يجب إدارتها.

2- آلية تسمية للوثائق.

3- تحديد مسؤوليات إجراءات إدارة التشكيلات.

4- تعريف سياسات التغيير وإدارة الإصدارات.

5- تعريف سجلات قاعدة معطيات إدارة التشكيلات التي يجب استخدامها.

6- تحديد الأدوات المساعدة التي يجب استخدامها وطريقة استخدامها.

للأدوات المساعدة في هندسة البرمجيات CASE tools أهمية خاصة في إدارة التشكيلات. فيمكن م-ن خلالها تطبيق معايير وإجراءات معرفة سلفاً. تتوفر الكثير من الأدوات في هذا المجال منها م-ا ه-و مستقل ومنها ما ينتمي لمنصة عمل متكاملة تغطي جميع مراحل الإجرائية البرمجية. تهتم إدارة الإصدارات بمجموعة من المهام مثل تحديد الإصدارات وتسجيل ال-سحوب المة-وفرة م-ن النظام:

- 1- وضع آلية ترقيم لإصدارات النظام.
- 2- التخطيط لمواعيد إنتاج إصدارات جديدة من النظام.
- 3- ضمان تطبيق إجراءات وأدوات إدارة الإصدارات والسحوب.
- 4- التخطيط للسحوب الجديدة وتوزيعها.

الإصدار هو نسخة جديدة من النظام تختلف وظيفياً عن النسخ المتوفرة سابقاً. أما عند المحافظة على الوظائف وتغيير النواحي غير الوظيفية فيسمى ذلك متغيراً **variant**. أما السحب فهو نسخة من النظام توزع على المستخدمين من خارج فريق التطوير.

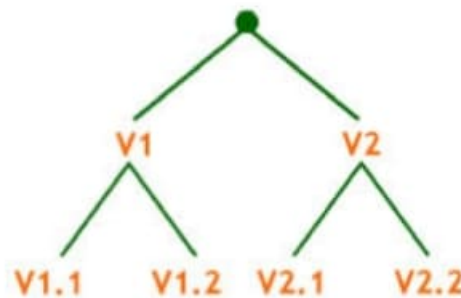
عندما يتألف النظام من مجموعة كبيرة من المكونات التي تتألف بدورها من مكونات جزئية وفرعية على أكثر من مستوى تكون هنالك حاجة كبيرة لإدارة إصدارات النظام ومعرفة مرجعية كل مكونة جزئية إلى الإصدار التي تنتمي إليه. يجب أن تعرف إجراءات تحديد الإصدارات طريقة واضحة لتحديد إصدارات المكونات. هنالك ثلاثة تقنيات أساسية لتحديد المكونات:

- 1- ترقيم الإصدارات.
- 2- التحديد بالاعتماد على الواصفات.
- 3- التحديد الموجه بالتغييرات.

تقنيات تحديد المكونات

1- ترقيم الإصدارات

آلية تسمية تستخدم اشتقاق خطي للأسماء ... V1, V1.1, V1.2, V2.1, ... ينتج عنها هرمية من الإصدارات على شكل شجرة.



2- التحديد بالاعتماد على الوصفات

يمكن إرفاق واصفات مع كل إصدار بحيث يتحدد الإصدار بمجموعة من الوصفات من الأمثلة ع- ن هذه الوصفات: التاريخ والمبرمج ولغة البرمجة والزيون والحالة ... يعطي ذلك مرونة أكبر من الترقيم الخطي لكنه يمكن أن يولد مشاكل في فريدة النسخة. يجب أن يجري اختيار مجموعة محددة من الوصفات بحيث يمكن تحديد كل إصدار بصورة وحيدة. من الفوائد الهامة لهذه الطريقة أنها تدعم إجراء استعلامات فيمكن مثلاً البحث عن أحدث إصدار بلغة جافا- ١.٥- ن نظام ما أو إجراء استعلام من الشكل:

AC3D (language =Java, platform = XP, date = Jan 2003)

3- التحديد الموجه بالتغييرات

يجمع بين الإصدارات والتغييرات التي أجريت للحصول عليها. يستخدم للأنظمة ولا يستخدم للمكونات. كل تغيير مقترح يتضمن مجموعة تغييرات يجب أن تُطبق بالتسلسل لتنجيذه. فنتج إصدارات متعددة كل منها يحتوي مجموعة من التغييرات.

يجب أن تتضمن السحوبات جميع التغييرات خاصة تلك المتعلقة بتصحيح أخطاء اكتشافها المستخدم أو المتعلقة بتغيير العتاد المادي وبيئة التشغيل. كما يجب أن تتضمن وظائف جديدة للنظام. تم إدارة السحوب بتقرير الوقت المناسب لإصدار سحب ما (لتحويل إصدار ما إلى سحب).

تتضمن السحوبات إضافة إلى البرامج التنفيذية ملفات التشكيلات التي تتضمن كيفية تشكيل البرامج-ات وإرسائها، وملفات المعطيات اللازمة لعمليات النظام، وبرنامج إرساء، ووثائق الكترونية ومطبوعة للمساعدة، ولوازم الحزم البرمجية والدعايات المرافقة لمنتجات أخرى ذات علاقة به- التطبيق. يجري إصدار السحوبات على أقراص ضوئية (CD, DVD) أو كم ملفات قابلة للتحميل عن طريق الويب.

يجب أن لا يفترض إدارة السحوب أن السحوب السابقة قد جرى قبولها وتعتمد على مكونات منها- ١ لسحب جديد، بل يجب أن يتضمن السحب الجديد كافة الملفات اللازمة للإرساء. يمكن أن لا يرغب المستخدم باقتناء سحب جديد من نظام ما لكونه مكثفي بالنظام الحالي ولا يحتاج للوظائف الجديدة المتوفرة في السحب الجديد.

إن تحضير وتوزيع سحب جديد من نظام ما هو إجرائية مكلفة لذلك يجب دراسة هذه العملية جيداً واتخاذ قرار مناسب بشأنها. يمكن لعدة عوامل أن تؤثر على هذا القرار مثل: الجودة التقنية للنظام، والمنافسة، ومتطلبات التسويق، وطلبات التغيير من قبل الزبائن.

تتوفر أدوات خاصة بإدارة الإصدارات والسحوبات تسمح بترقيم آلي للإصدارات (النظام والمكونات)،
وتسمح بتخزين الاختلافات بين الإصدارات، وتسجيل تاريخي للإصدارات وأسبابها، وتساعد في فصل
أعمال التطوير على الإصدارات المختلفة.